Inspection Factorization for Hardware-Efficient 2D Fast Convolution

Társio Onofrio da Silva and Fernando Gehm Moraes
School of Technology, Pontifical Catholic University of Rio Grande do Sul – PUCRS – Porto Alegre, Brazil
tarsio.onofrio@edu.pucrs.br, fernando.moraes@pucrs.br

***Abstract*—Fast convolution is widely used to reduce the arithmetic cost of small-kernel CNN convolutions. However, its hardware efficiency depends not only on the number of transform-domain multiplications, but also on the complexity of the transform stages. This work proposes Inspection Factorization (*IF*) as an alternative to conventional Winograd and Toom–Cook schemes for 3×3 convolution under a unified RTL architecture. *IF* is compared to a straightforward MAC-based baseline, referred to as *Naïve*, under identical workload and clock constraints at two operating points: *balanced*, corresponding to the best area–performance trade-off, and *maxMAC*, corresponding to the maximum number of MAC units used by the *IF* in this work. Relative to *Naïve*, *IF* achieves a performance gain of 69% to 83% for both operating points. The corresponding area variation ranges from a 2% reduction to a 38% increase, while power consumption increases by 44% to 88%. Despite this power penalty, *IF* reduces energy consumption by 57% to 69%. Compared with other fast-convolution alternatives, *IF* offers an intermediate trade-off between performance and implementation cost, with lower transform overhead than larger Toom–Cook configurations and higher performance than smaller Winograd-based alternatives.**

***Index Terms*—Fast Convolution, Inspection Factorization, Winograd, Toom–Cook, Hardware Acceleration.**

I. Introduction

Convolution is one of the main computational bottlenecks in CNN inference because it is repeatedly applied across many channels and spatial positions, directly affecting the throughput and energy consumption of hardware accelerators [1]–[3]. This issue is particularly important in embedded and edge systems, where area, power, and memory constraints limit the range of feasible architectural optimizations [4, 5].

Fast convolution algorithms are widely adopted in this scenario because they reduce the number of multiplications required to generate an output tile. Among them, Winograd and Toom-Cook constructions are the most frequently used in the literature, especially for small kernels [6]–[9]. However, arithmetic optimality alone does not guarantee hardware efficiency. As the output tile size increases, the transform matrices often introduce nontrivial constants, which raise the cost of additions, constant multiplications, storage, and control. In practice, this transform overhead may offset the savings obtained from reducing element-wise multiplications [10].

Most CNN hardware proposals explore only a limited subset of fast-convolution options, mainly minimal Winograd tiles and small Toom-Cook instances [11]–[19]. Even when fast methods are incorporated into broader frameworks or template-based accelerators, the practical design space is usually restricted to a few predefined configurations selected on a per-layer basis [20]–[22].

Recent configurable generators and specialized Winograd-domain accelerators [23]–[28] expand but still constrain the design space to a few algorithmic variants. These works, however, still evaluate a few algorithmic variants under architecture-specific assumptions, leaving alternative constructions underexplored in a controlled RTL comparison. In particular, Inspection Factorization (IF), although mentioned in the literature related to Winograd methods [29], has not been systematically evaluated in CNN hardware architectures. IF is a factorization strategy for fast convolution, rather than a distinct convolution model, and can be mapped onto the same transform-based formulation used by Winograd methods.

This paper proposes IF as a hardware-oriented alternative for 3×3 two-dimensional convolution. The focus on 3×3 is motivated by their prevalence in standard CNN benchmarks, where they constitute the vast majority of convolutional layers; the method generalizes to larger kernels, but the 3×3 case presents the most relevant size for embedded accelerators. The key idea is to use IF to replace multiplications by general constants while preserving a transform structure compatible with a unified fast-convolution datapath. To ensure a fair comparison, all algorithms are implemented using the same RTL template; therefore, differences in performance, area, power, and energy stem primarily from the convolution algorithm itself rather than from unrelated architectural modifications. The main contribution of this work is to show that IF defines an intermediate design point between minimal Winograd and larger Toom-Cook configurations[1].

The remainder of this paper is organized as follows. Section II summarizes the fast-convolution formulation adopted in this work. Section III discusses the trade-offs that motivate the IF proposal. Sections IV and V present the IF architecture and results. Section VI concludes this paper.

II. Fast Convolution Background

Fast convolution can be expressed as a sequence of linear transforms, element-wise products, and an inverse transform. In this work, all algorithms are implemented and evaluated within the same Winograd-style formulation [6], so that the datapath structure remains comparable across algorithms even when coefficients and multiplication counts differ. Appendix A consolidates the mathematical formulation.

[1]Project repositories: https://github.com/tarsioonofrio/fast-convolution-rtl, https://github.com/tarsioonofrio/FastConv_SystemVerilog

Algorithm selection affects not only the element-wise multiplication count (Equation (3) in Appendix A) but also the complexity of the transform networks, the amount of intermediate data stored, and the number of control steps during the multiplication step. The relevant cost, therefore, includes the combined overhead of transforms, storage structures, and multipliers, not only the arithmetic minimum.

III. Proposed Algorithms and Operation Cost

For 3×3 kernels, increasing the output tile size can reduce the number of multiplications associated with transform-domain element-wise products. However, this reduction usually comes at the cost of more expensive transforms. In larger Toom-Cook instances [7, 30, 31], the transform matrices include constants other than 0, ± 1, and powers of two, which increases hardware cost. Even when such constant products are implemented using shift-and-add networks, they still increase area, logic depth, routing complexity, and switching activity.

Inspection Factorization (IF) [10] offers a different trade-off: instead of minimizing transform-domain variable multiplications, it derives a convolution form in which the transform and inverse stages require only additions and subtractions, eliminating constant multipliers. The IF construction used here follows the description in 1D [6, 8, 10, 29], yielding 36 variable multiplications in the 2D nested form; further derivation details appear in the companion repository. This property is significant when transform cost is non-negligible: although IF may require more variable multiplications than some Toom-Cook configurations, the elimination of constant multipliers simplifies the transform networks and reduces switching activity.

Table I summarizes the operation count for the evaluated algorithms. IF matches the variable multiplication count of $TC_{4\times4}$ while eliminating all 36 constant multiplications and requiring only 24 additions, compared to 56 for $TC_{4\times4}$.

TABLE I
Operation-count comparison among the evaluated convolution algorithms.

Algorithm	Toom-Cook			*IF*
Output size (M)	2×2	3×3	4×4	3×3
Additions	12	34	56	24
Constant Multiplications	0	18	36	0
Variable Multiplications	16	25	36	36
All Multiplications	16	43	72	36

If removing constant multipliers is sufficiently beneficial at the RTL level, IF should provide a lower combined power-performance-area cost than the variable multiplication count alone would predict.

IV. Hardware Implementation

To evaluate this hypothesis at the RTL level, all algorithms are implemented within the same parameterizable architecture. This choice ensures that differences in power, performance, and area arise from the convolution method itself rather than from unrelated changes in datapath structure or control design.

The convolution core consists of three main blocks that implement Equation (3) and are organized as a pipeline: *Transform*, executed in one clock cycle (*cc*); *Hadamard*, requiring multiple *cc*; and *Inverse*, executed in one *cc*.

A local register bank stores the active tile and its intermediate values across these blocks. The transformed filter coefficients are assumed to be precomputed offline and supplied directly to the *Hadamard* block[2].

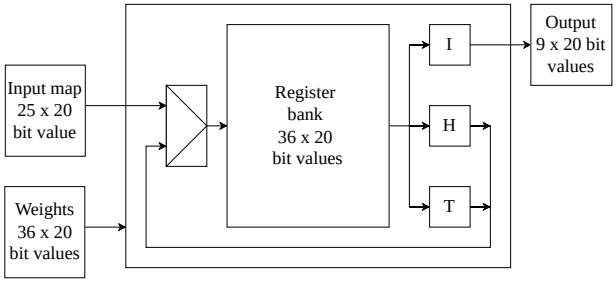

Fig. 1. IF example of block-level organization of the convolution core, showing the register bank and the Transform (T), Hadamard (H), and Inverse (I) pipeline stages. The register bank stores all intermediate results across stages.

The *Transform* and *Inverse* blocks are implemented as constant-coefficient linear networks that use only shift-and-add. Their coefficients depend on the selected fast-convolution algorithm, but their architectural role is unchanged across all configurations. Consequently, substituting Toom-Cook or IF for the minimal Winograd filter is equivalent to replacing the transform coefficient matrices while leaving the datapath structure intact.

The Hadamard stage is implemented using a parameterizable number of shared multipliers to perform the variable multiplications. Rather than instantiating all multiplications in parallel, the architecture executes them across one or more internal states. This organization enables the same convolution method to be evaluated across different multiplier budgets, revealing the trade-off between throughput and silicon cost. The degree of multiplier parallelism is therefore an architectural parameter, while the total number of variable multiplications is an algorithmic property fixed by the chosen algorithm.

The set of evaluated algorithms is summarized in Table II. These configurations include the naïve baseline, representative Winograd/Toom-Cook variants, and the proposed IF. Each algorithm is defined by its input and output tile sizes, which determine both the transform structure and the number of tiles required to process a feature map.

Table III lists, for each algorithm, the number of variable multiplications together with representative MAC configurations and their corresponding multiplication-state counts. The number of MACs must divide the total multiplication count evenly to avoid underutilization. Two specific configurations, *balanced* (best area–performance trade-off) and *maxMAC* (maximum parallelism evaluated in this work), are used as reference points throughout Section V.

[2]A short clip illustrates the transform–multiply–inverse pipeline and register-bank reuse: https://youtu.be/WgymWFtuF30.

TABLE II
EVALUATED ALGORITHMS WITH THE SIZES OF THE INPUT AND THE OUTPUT.

Symbol	Algorithm	Input	Output
Naïve	Standard convolution	3×3	1
$TC_{2\times2}$	Toom Cook or Winograd Minimal filter	4×4	2×2
$TC_{3\times3}$	Toom Cook	5×5	3×3
$TC_{4\times4}$	Toom Cook	6×6	4×4
IF	**Inspection Factorization**	5×5	3×3

TABLE III
EVALUATED ALGORITHMS, THE NUMBER OF VARIABLE MULTIPLICATIONS AND AN EXAMPLE CONFIGURATIONS OF EACH ONE.

Symbol	Multiplications	Example
$TC_{2\times2}$	16	2 MACs and 8 multiplication states
$TC_{3\times3}$	25	5 MACs and 5 multiplication states
$TC_{4\times4}$	36	12 MACs and 3 multiplication states
IF	**36**	**18 MACs and 2 multiplication states**

V. RESULTS

We conduct the experiments on synthesized RTL designs targeting a 28 nm technology node under a 500 MHz clock constraint. We measure area after logic synthesis as the total cell area. For power estimation, we simulate the synthesized netlist and back-annotate the resulting switching activity into the synthesis environment. Inputs are sampled from a normal distribution, quantized to 8 bits, and processed with a 20-bit internal word size over a 32×32 input feature map. We compute energy as the product of average power and execution time.

Figure 2 presents the performance of the evaluated algorithms in clock cycles. Performance does not scale linearly with multiplier count: for each algorithm, execution time initially decreases as parallelism increases, but saturates once the fixed-latency Transform and Inverse stages dominate. The *balanced* operating point corresponds to the knee of each curve, where additional multipliers yield diminishing throughput returns relative to their area cost.

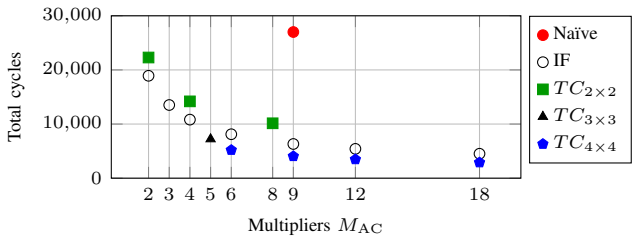

Fig. 2. Execution time in clock cycles as a function of MAC count. Performance saturates beyond 6 MACs for IF and $TC_{4\times4}$, marking the *balanced* operating point.

Figure 3 presents the cell area of the evaluated algorithms. A lower multiplication count does not imply smaller area: larger transform-domain structures require additional registers and more complex linear networks, so $TC_{4\times4}$ incurs 62% more area than the naïve baseline despite its higher throughput. The minimal Winograd filter achieves a lower area owing to its compact transform structure; IF trades constant multipliers for additional variable multiplications, keeping area close to the naïve baseline while achieving higher throughput.

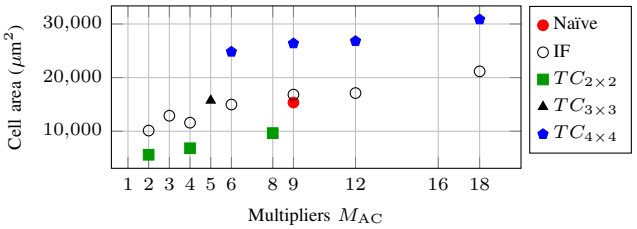

Fig. 3. Total cell area per configuration (μm^2) as a function of the multiplier count.

As shown in Figure 4, power follows a trend similar to area, as it depends on the amount of logic and storage that are active during execution. In general, higher-parallelism configurations consume more power, and more complex transforms increase switching activity. IF's binary transform coefficients $(0, \pm 1)$ avoid the shift-and-add networks required by $TC_{3\times3}$ and $TC_{4\times4}$, helping keep IF power below larger Toom-Cook configurations while still increasing power relative to Naïve. Consequently, a shorter execution time does not guarantee lower energy if the accompanying increase in area and switching activity is excessive.

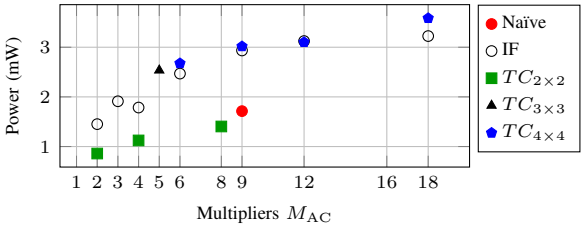

Fig. 4. Power consumption (mW) as a function of the multiplier count.

Figure 5 shows the combined effect of these trade-offs on energy. All fast-convolution algorithms reduce execution time relative to the Naïve baseline, but they do so with different trade-offs in cost. The $TC_{2\times2}$ favors efficiency-oriented operating points by combining low transform cost with cycle reduction. Larger Toom-Cook configurations favor throughput-oriented operating points, often at the expense of higher area and power. IF occupies an intermediate point: it reduces energy relative to Naïve, with cycle reduction while avoiding the area increase of $TC_{4\times4}$.

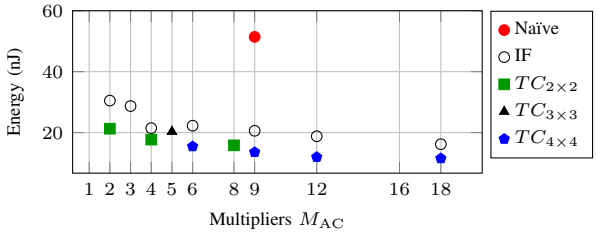

Fig. 5. Energy evaluation as a function of the multiplier count.

Table IV summarizes the normalized comparison across the evaluated algorithms. IF does not lead in any single metric, but at the *balanced* point, its combination of performance gain, area, and energy reduction distinguishes it from both extremes: $TC_{2\times2}$ saves more area but is 7% slower, while $TC_{4\times4}$ is 13% faster but incurs more power and 60% more area. At the *maxMAC* point, IF also offers a favorable trade-off relative to $TC_{4\times4}$: it is 6% slower in performance and 7% lower in energy consumption, but dissipates 21% less power and improves the normalized area cost by 63%.

TABLE IV
NORMALIZED COMPARISON WITH RESPECT TO THE NAÏVE BASELINE.
Positive values indicate improvement over Naïve (higher performance or lower area, power, and energy), while negative values indicate degradation.

	Balanced				
Name	MACs	Perf.	Area	Power	Energy
$TC_{2\times2}$	8	62%	37%	18%	69%
$TC_{3\times3}$	5	72%	-3%	-48%	60%
$TC_{4\times4}$	6	76%	-62%	-56%	70%
IF	**6**	**69%**	**2%**	**-44%**	**57%**
	Max MAC number				
Name	MACs	Perf.	Area	Power	Energy
$TC_{4\times4}$	18	89%	-101%	-109%	77%
IF	**18**	**83%**	**-38%**	**-88%**	**69%**

IF expands the fast-convolution design space by offering a throughput advantage over $TC_{2\times2}$ without incurring the area overhead of $TC_{4\times4}$, thereby occupying a distinct intermediate point that prior hardware surveys have left uncharacterized.

VI. CONCLUSION

This work evaluated IF as a hardware-oriented alternative for 3×3 two-dimensional fast convolution. By implementing all candidates within the same RTL template and measuring area, power, and energy at the 28 nm node, we show that transform complexity, not only variable multiplication count, is a decisive factor in hardware cost.

IF's primary advantage is the elimination of constant multiplications during the transform and inverse stages, which reduces switching activity, area overhead, and simplifies routing compared to larger Toom-Cook instances.

Fast-convolution hardware evaluation should therefore account for the full transform cost, not only the Hadamard-stage multiplication count. IF is a practical instantiation of this principle for accelerators where minimal Winograd configurations do not meet throughput requirements and the area overhead of large Toom-Cook tiles is unacceptable.

As future work, we plan to extend this comparison to larger CNN workloads, broader quantization studies, and more aggressive dataflow optimizations that reduce internal register-bank usage and expose additional overlap between transform and multiplication stages.

APPENDIX A
CONVOLUTION EQUATIONS AND MATRIX FORMULATION

Naïve convolution is the direct spatial-domain reference used throughout this work. In one dimension, each output sample is computed as the sum of pairwise products between the input window and the kernel, as shown in Equations (1) and (2)[3].

$$s[i] = (d * g)[i] = \sum_{k=0}^{W-1} d[i+k] \cdot g[k] \tag{1}$$

$$s[i_1 i_2] = (d * *g)[i_1 i_2] = \textstyle\sum_{k_1=0}^{W_1-1} \sum_{k_2=0}^{W_2-1} d[i_1 - k_1, i_2 - k_2] g[k_1 k_2] \tag{2}$$

In two dimensions, the same operation is extended across rows and columns, as shown in Equation (3).

$$s = A^T \underbrace{\left[\underbrace{(QBg)}_{\text{G: transformed weights}} \overbrace{\odot}^{\substack{\text{element-wise} \\ \text{multiplications}}} \underbrace{(C^T d)}_{D\text{: Transform}} \right]}_{S\text{: Hadamard}} \implies s = \underbrace{A^T S}_{\text{Inverse}} \tag{3}$$

where: d is the input feature map (IFMAP), g are the weights, and C, B, A, and Q are transform matrices, and s is the output feature map (OFMAP).

Equations (4) and (5) show the Toom-Cook matrices used in this work.

$$A = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & -1 \\ 0 & 1 \end{bmatrix} \quad B = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & -1 & 1 \\ 0 & 0 & 1 \end{bmatrix} \quad C = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & -1 & -1 \\ 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad Q = \begin{bmatrix} -1 \\ \frac{1}{2} \\ \frac{1}{2} \\ 1 \end{bmatrix} \tag{4}$$

$$A = B = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & -1 & 1 \\ 1 & 2 & 4 \\ 0 & 0 & 1 \end{bmatrix} \quad C = \begin{bmatrix} 2 & 0 & 0 & 0 & 0 \\ -1 & -2 & 2 & -1 & 2 \\ -2 & -1 & -3 & 0 & -1 \\ 1 & 1 & 1 & 1 & -2 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix} \quad Q = \begin{bmatrix} \frac{1}{2} \\ -\frac{1}{2} \\ -\frac{1}{6} \\ \frac{1}{6} \\ 1 \end{bmatrix} \tag{5}$$

For the proposed Inspection Factorization, Equation (6) shows the corresponding matrices. All non-zero entries are ±1, ensuring that both the transform and inverse stages reduce to additions and subtractions with no constant multiplications required.

$$A = B = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} \quad C = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ -1 & -1 & 0 & 1 & 0 & 0 \\ -1 & 1 & -1 & 0 & 1 & 0 \\ 0 & -1 & -1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} \quad Q = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} \tag{6}$$

In the nested 2D form [32], the 1D transforms are applied successively along each tile dimension, resulting in the 2D expression shown in Equation (7) [9]. This preserves a common transform-Hadamard-inverse structure in hardware while changing only matrix coefficients, and multiplication counts across algorithm families[4].

$$s = A^T \Big\{ \left[(QB) g (QB)^T \right] \odot (C^T d C) \Big\} A \tag{7}$$

[3]A short clip of standard Naïve convolution: https://youtu.be/itwrJ3AFF1M.
[4]A short clip of IF nested: https://youtu.be/J5lgvCpZV64. A second clip shows a 2D fast-convolution sliding window: https://youtu.be/Toe6KPWF5RQ.

ACKNOWLEDGMENTS

This work was financed in part by Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq), 305621/2024-6; Fundação de Amparo à Pesquisa do Estado do Rio Grande do Sul (FAPERGS), 23/2551-0002200-1.

REFERENCES

[1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with Deep Convolutional neural networks," *Communications of the ACM*, pp. 84–90, 2017, https://doi.org/10.1145/3065386.

[2] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge Academic Press, 2016, https://doi.org/10.1017/9781108564175.032.

[3] W. J. Dally, Y. Turakhia, and S. Han, "Domain-specific hardware accelerators," *Communications of the ACM*, pp. 48–57, 2020, https://doi.org/10.1145/3361682.

[4] A. Shawahna, S. M. Sait, and A. El-Maleh, "FPGA-Based Accelerators of Deep Learning Networks for Learning and Classification: A Review," *IEEE Access*, pp. 7823–7859, 2019, https://doi.org/10.1109/access.2018.2890150.

[5] D. Moolchandani, A. Kumar, and S. R. Sarangi, "Accelerating CNN Inference on ASICs: A Survey," *Journal of Systems Architecture*, pp. 1–26, 2021, https://doi.org/10.1016/j.sysarc.2020.101887.

[6] S. Winograd, *Arithmetic Complexity of Computations (CBMS-NSF Regional Conference Series in Applied Mathematics)*. SIAM Publication Library, 1987, https://doi.org/10.1137/1.9781611970364.

[7] R. Tolimieri, M. An, and C. Lu, *Algorithms for Discrete Fourier Transform and Convolution*. Springer-Verlag, 2013, https://doi.org/10.1007/978-1-4757-2767-8.

[8] K. K. Parhi, *VLSI Digital Signal Processing Systems: Design and Implementation*. Wiley-Interscience, 1999.

[9] A. Lavin and S. Gray, "Fast Algorithms for Convolutional Neural Networks," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 4013–4021, https://doi.org/10.1109/cvpr.2016.435.

[10] R. E. Blahut, *Fast Algorithms for Signal Processing*. Cambridge Academic Press, 2010, https://doi.org/10.1017/cbo9780511760921.

[11] Y. Liang, L. Lu, Q. Xiao, and S. Yan, "Evaluating fast algorithms for convolutional neural networks on FPGAs," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, p. 857–870, 2020, https://doi.org/10.1109/tcad.2019.2897701.

[12] S. Kala, B. R. Jose, J. Mathew, and S. Nalesh, "High-Performance CNN Accelerator on FPGA Using Unified Winograd-GEMM Architecture," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, p. 2816–2828, 2019, https://doi.org/10.1109/tvlsi.2019.2941250.

[13] Q. Xiao, Y. Liang, L. Lu, S. Yan, and Y.-W. Tai, "Exploring Heterogeneous Algorithms for Accelerating Deep Convolutional Neural Networks on FPGAs," in *Design Automation Conference (DAC)*, 2017, pp. 1–6, https://doi.org/10.1145/3061639.3062244.

[14] Y. Zhao, D. Wang, and L. Wang, "Convolution accelerator designs using fast algorithms," *Algorithms*, p. 112, 2019, https://doi.org/10.3390/a12050112.

[15] L. Lu, Y. Liang, Q. Xiao, and S. Yan, "Evaluating fast algorithms for convolutional neural networks on fpgas," in *Proc. - IEEE Annu. Int. Symp. Field-Program. Cust. Comput. Mach., FCCM*, 2017, p. 101–108, https://doi.org/10.1109/fccm.2017.64.

[16] S. Kala, D. Paul, B. Jose, and S. Nalesh, "Design Space Exploration of Convolution Algorithms to Accelerate CNNs on FPGA," in *International Symposium on Embedded Computing and System Design (ISED)*, 2018, pp. 21–25, https://doi.org/10.1109/ised.2018.8704043.

[17] J. Yu, Y. Hu, X. Ning, J. Qiu, K. Guo, Y. Wang, and H. Yang, "Instruction Driven Cross-layer CNN Accelerator with Winograd Transformation on FPGA," in *International Conference on Field Programmable Technology (ICFPT)*, 2020, pp. 227–230, https://doi.org/10.1109/fpt.2017.8280147.

[18] C. Zhuge, X. Liu, X. Zhang, S. Gummadi, J. Xiong, and D. Chen, "Face Recognition with Hybrid Efficient Convolution Algorithms on FPGAs," in *Great Lakes Symposium on VLSI (GLSVLSI)*, 2018, pp. 123–128, https://doi.org/10.1145/3194554.3194597.

[19] A. Ahmad and M. A. Pasha, "Ffconv: An fpga-based accelerator for fast convolution layers in convolutional neural networks," *ACM Trans. Embed. Comput. Syst.*, p. 15:1–15:24, 2020, https://doi.org/10.1145/3380548.

[20] R. DiCecco, G. Lacey, J. Vasiljevic, P. Chow, G. Taylor, and S. Areibi, "Caffeinated FPGAs: FPGA Framework For Convolutional Neural Networks," in *International Conference on Field-Programmable Technology (FPT)*, 2016, pp. 265–268, https://doi.org/10.1109/fpt.2016.7929549.

[21] H. Ye, X. Zhang, Z. Huang, G. Chen, and D. Chen, "HybridDNN: A Framework for High-Performance Hybrid DNN Accelerator Design and Implementation," in *ACM/IEEE Design Automation Conference (DAC)*, 2020, pp. 1–6, https://doi.org/10.1109/dac18072.2020.9218684.

[22] S. Kala, J. Mathew, B. R. Jose, and S. Nalesh, "UniWiG: Unified Winograd-GEMM Architecture for Accelerating CNN on FPGAs," in *International Conference on VLSI Design (VLSID)*, 2019, pp. 209–214, https://doi.org/10.1109/vlsid.2019.00055.

[23] M. Li, P. Li, S. Yin, S. Chen, B. Li, C. Tong, J. Yang, T. Chen, and B. Yu, "WinoGen: A highly configurable winograd convolution IP generator for efficient CNN acceleration on FPGA," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6, https://doi.org/10.1145/3649329.3657392.

[24] K. Xie, Y. Lu, X. He, D. Yi, H. Dong, and Y. Chen, "Winols: A large-tiling sparse winograd CNN accelerator on FPGAs," *ACM Transactions on Reconfigurable Technology and Systems*, pp. 1–24, 2024, https://doi.org/10.1145/3643682.

[25] J. Li, Y. Liang, Z. Yang, and X. Li, "An efficient convolutional neural network accelerator design on FPGA using the layer-to-layer unified input winograd architecture," *Electronics*, 2025, https://doi.org/10.3390/electronics14061182.

[26] C. Yang, Y. Wang, X. Wang, and L. Geng, "Wra: A 2.2-to-6.3 tops highly unified dynamically reconfigurable accelerator using a novel winograd decomposition algorithm for convolutional neural networks," *IEEE Transactions on Circuits and Systems I: Regular Papers*, p. 3480–3493, 2019, https://doi.org/10.1109/tcsi.2019.2928682.

[27] X. Liu, Y. Chen, C. Hao, A. Dhar, and D. Chen, "WinoCNN: Kernel Sharing Winograd Systolic Array for Efficient Convolutional Neural Network Acceleration on FPGAs," in *IEEE International Conference on Application-specific Systems, Architectures and Processors (ASAP)*, 2021, pp. 258–265, https://doi.org/10.1109/asap52443.2021.00045.

[28] X. Wang, C. Wang, J. Cao, L. Gong, and X. Zhou, "Winonn: Optimizing fpga-based convolutional neural network accelerators using sparse winograd algorithm," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, p. 4290–4302, 2020, https://doi.org/10.1109/tcad.2020.3012323.

[29] T. O. Da Silva and F. G. Moraes, "Fast and energy-efficient 2d convolution: Inspection-based methods for hardware acceleration," in *2025 38th SBC/SBMicro/IEEE Symposium on Integrated Circuits and Systems Design (SBCCI)*, 2025, pp. 1–5, https://doi.org/10.1109/sbcci66862.2025.11218694.

[30] S. Winograd, "Some bilinear forms whose multiplicative complexity depends on the field of constants," *Mathematical systems theory*, p. 169–180, 1976, https://doi.org/10.1007/bf01683270.

[31] D. G. Myers, *Digital Signal Processing: Efficient Convolution and Fourier Tranform Techniques*. Prentice Hall, 1990.

[32] S. Winograd, "On computing the discrete fourier transform," *Mathematics of Computation*, p. 175–199, 1978, https://doi.org/10.2307/2006266.